P2992R0
Attribute [[discard]] and attributes on expressions

Published Proposal, 2023-10-09

Author:
Giuseppe D'Angelo

Audience:
SG17, EWG, SG22

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We propose a new standard attribute, `[[discard]]`, to express the explicit intent of discarding the result of an expression. We also propose to allow attributes on expressions.

Table of Contents

1 Changelog

2 Motivation and Scope

2.1 Existing discarding strategies

2.1.1 Cast to `void`

2.1.2 Assignment to `std::ignore`

2.2 A new attribute

3 Design Decisions

3.1 Is `[[discard]]` an attribute on expressions or statements?

3.2 Attributes on expressions: approach 1

3.2.1 Problems

3.3 Attributes on expressions: approach 2

3.4 Can `void` be discarded?

3.5 What should happen if `[[discard]]` is applied to an expression which isn't a discarded-value expression?

4 Impact on the Standard

5 Technical Specifications

5.1 Proposed wording

6 Acknowledgements

References

Informative References

§ 1. Changelog

- R0
  - First submission

§ 2. Motivation and Scope

[P0068R0] (and its subsequent [P0189R1] and [P1301R4]) introduced the `[[nodiscard]]` attribute in C++17; [WG14-N2267] introduced the same attribute in C23.

The main use case for the `[[nodiscard]]` attribute is to mark functions (and types returned by functions) whose return value is deemed important, and discarding it can have important consequences.

Examples of such functions include:

- **Pure functions**, that is, functions with no side effects, that are just called for their return value (example: `std::vector::empty()`). If the return value is not examined, it does not make sense to call the function at all.
  This is not merely a performance issue, but also a correctness one: functions and types that are poorly named (again, `empty()`) may mislead the user into thinking that the function actually does something (it *empties* the vector).

- **Functions that return error codes** or similar values that *must be examined* (and therefore used) for the calling code to be correct.
  In this case, discarding the return value constitutes a logical bug in user code, and encouraging a warning from the implementation should make the user aware of their mistake.
- **Functions that return unmanaged objects** that must be reclaimed in a special way (example: `std::allocator<T>::allocate`, that returns a pointer allocated with `operator new`).
  For these functions, accidentally discarding the return value incurs in resource leaks.
- **Functions that return manager objects** that should be kept alive; for instance, RAII managers that need to be kept alive in the block in which the function is invoked.
  Discarding the return value does not constitute any resource leak *per se*, but code "close" to the function call (e.g. in the same block) may operate under the false assumption that the manager (and thus the resource) is still alive/acquired.
  Examples of these functions are (after [P1771R1]) constructors of lock managers, where the constructed object itself should not get discarded, otherwise the following code would operate without the necessary mutex protection.

A "discarded return value" is formally defined as a *discarded-value expression* in [expr.context]/2. They are:

- expressions appearing as *expression-statement* productions ([stmt.expr]/1);
- the left-side expression of the builtin comma operator ([expr.comma]/1);
- expressions explicitly converted to (cv-)void ([expr.static.cast]/6).

Still: in some scenarios it is useful to expressely discard the result of an expression, without having the implementation generating a warning in that case. Examples include:

- **Testing**. We may want to call a nodiscard function in order to e.g. perform smoke testing, make sure that wide-contract functions don't crash, and similar. In these cases we are simply not interested in the return value, yet we do not want a warning.
- **Partial domains**. A nodiscard function that returns an error code may also document that it will never fail under certain conditions (for instance, when certain values are passed as parameters). If the user is already checking that the parameters are in the non-failing domain, there is no need to use the return value; it can be safely discarded.
  One could argue that the code could still consume the error code, and for instance `assert` on it; but, in general, we may simply not be interested in *enforcing a contract* on the function. This is akin to testing the function itself; in other words, to verify that the function matches its

documented semantics. This is not the job of the user of a function, but of the implementor of the function.

- **Legacy**. A function may return a status code to indicate success or failure; then, later, its implementation evolves in a way such that it never fails (example: `pthread_once` on Linux). Still, the function may decide to keep returning the status code to preserve API and ABI compatibility. The return value can always be safely discarded as it's meaningless.

This paper proposes a standard way to express the user's intent to discard the value of an expression. Such a way is currently missing, and workarounds are employed instead.

§ 2.1. Existing discarding strategies

There are currently two main strategies for discarding a value returned from a `[[nodiscard]]` function without producing a warning by the implementation.

§ 2.1.1. Cast to `void`

[dcl.attr.nodiscard]/4 states:

> Appearance of a nodiscard call as a potentially-evaluated discarded-value expression ([expr.prop]) is discouraged unless explicitly cast to `void`. Implementations should issue a warning in such cases.

This means that an explicit cast to `void` can be used to suppress the nodiscard warning:

```
[[nodiscard]] int f();

f();         // warning here
(void) f();  // no warning
void(f());   // no warning
```

Using a cast to suppress the warning has several major shortcomings:

- It is a total abuse of notation. A cast notation is used, although one does not intend perform any type conversion; the code is just using an "arcane" language rule (any expression can be converted to *cv* `void`, [expr.static.cast]/6), as well as the explicit leeway that `[[nodiscard]]` leaves, in order to suppress the warning.

- It is hard to explain to language novices why the cast works and what it means.
- It is hard/impossible to grep for in a codebase. Also, different codebases use slightly different spellings for the cast (`void(x)`, `(void)x`, etc.). As a workaround, some codebases hide the cast behind a macro, to better express the intent and make it greppable again.
- The very same notation is also used for `[[maybe_unused]]` semantics, that is, "this object may be unused in this scope". Again, some codebases hide this *different* intent with a different macro.
- The rationale for the discard can only be provided via comments, not in code, and therefore it's hard/impossible to enforce rules that mandate such a rationale in a codebase.

2.1.2. Assignment to `std::ignore`

It is possible to utilize `std::ignore` as a "sink" object:

```
[[nodiscard]] int f();

std::ignore = f(); // return value isn't discarded => no warning
```

Nitpicking, at the moment `std::ignore` is a facility related to tuples and `std::tie` (see [tuple.creation]); the code above is not *guaranteed* to work, although it does work on all major implementations. [P2968R0] aims at standardizing the current behavior, and therefore give precise semantics to the code above.

This solution is also "blessed" by the C++ Core Guidelines ([ES.48]), which favor it over the cast to `void`.

Still, we claim that this solution has a number of shortcomings:

- It is not usable by generic code, since it requires the right-hand side of the assignment to yield a value. If the right-hand side expression evaluates to `void`, the code becomes ill-formed.
- It is not compatible with C.
- `std::ignore` is a library solution. We believe a language solution to the problem would be better suited for what's exquisitely a language issue.
- It is relatively verbose compared to the cast to `void`.
- Again, the rationale as of why we want to discard the result value can only be provided in comments.

§ 2.2. A new attribute

We find both strategies suboptimal, and therefore in this paper we are proposing a novel one: the `[[discard]]` attribute, that complements `[[nodiscard]]`:

```
[[nodiscard]] int f();

f();                                // warning here
[[discard("just testing")]] f();  // no warning
```

Compared with the previous solutions:

- `[[discard]]` is symmetrical to `[[nodiscard]]` in usage and syntax: `[[nodiscard]]` is placed on the function/type declaration that we do not want callers to ignore; `[[discard]]` is placed on the expression whose result one expressly want to discard. This makes it straightforward to learn and understand.
- It supports a reason, expressed right in the attribute. While a C++ compiler would not use the reason itself (because it **won't** generate a warning), having a reason available is useful for users and other tooling. Specifically:
  - since the whole `[[nodiscard]]` mechanism is opt-in, if a function or a type is marked as such, it means that someone went the extra mile to warn us about a possible mistake we are making (possibly with a message, as in `[[nodiscard("reason")]]`). Therefore, we would have a *structured* way to justify why we think that discarding the result is appropriate;
  - having a reason can also be enforced by tooling (for instance, code checkers could ban the usage of `[[discard]]` *without* a reason).
- It works with generic code (one can discard `void`).
- It does not require to pull components of the Standard Library for suppressing a language warning.
- It is perfectly compatible with C, and we would welcome its adoption by WG14 as well.
- It is moderately more verbose than the cast to `void`.

§ 3. Design Decisions

§ 3.1. Is `[[discard]]` an attribute on expressions or statements?

Since `[[discard]]` is really meant to be applied on function calls, which are expressions, it sounds natural that it should be described as an attribute that appertains to expressions. In principle this would also allow for it to be applied on subexpressions and have a "localized" effect:

```
//  do not warn for discarding the result of f() nor g(),
//  but warn for discarding a() and/or b():

a(), [[discard]] f(1, ([[discard]] g(), 2), 3), b();
```

✧ ✧ ✧

Now, attributes on expressions are a novelty, because C++'s grammar does not yet support them. The grammar production for *expression* ([expr.comma]) is:

```
expression:
    assignment-expression
    expression , assignment-expression
```

The "obvious" modification of this production to introduce attributes could look like this:

```
expression:
    attribute-specifier-seq_opt assignment-expression  // not proposed!
    expression , assignment-expression
```

However **this modification clashes** with the position where we can declare attributes on *expression-statement*s productions ([stmt.pre], [stmt.expr]). This is the relevant production:

```
statement:
    attribute-specifier-seq_opt expression-statement
```

with

```
expression-statement:
    expression_opt ;
```

In other words, in code like:

```
[[discard]] f();  // the *statement* is discarded!
```

the grammar is already ruling that the attribute appertains to the **statement**, not to the **expression**! We do not want to modify the *statement* productions, as that would be a source incompatible change (for instance, attributes that can only appertain to statements would now be rejected).

On the other hand, the snippet above shows what is going to be the most common usage of `[[discard]]`: applied to a statement that contains a function call. **For this reason, we are proposing that the `[[discard]]` attribute should first and foremost be applicable to an *expression-statement*.**

If that expression contains multiple discarded-value expressions (by means of operator comma), the attribute will apply to them all, suppressing all their possible warnings:

```
[[discard]] a(), b(), c();   // don't generate discarding warnings
```

This leaves us with the more general case of discarding just an expression. This is a somehow "secondary" goal, that we are pursuing for completeness' sake, because the only practical applications of such a discarding mechanism that we can find exists in the context of using the builtin comma operator. We strongly believe that usages of the builtin comma operator should be frowned upon, except in corner cases where it's unpractical to use alternatives.

Nonetheless, we propose to introduce attributes on expressions. To this end, in this paper we are exploring two different approaches.

§ 3.2. Attributes on expressions: approach 1

Given the grammar clash described above, the only "room" we have left to attach an attribute to a general expression is on the **right hand side** of an *expression*. We can modify the *expression* production as follows:

```
expression:
    assignment-expression attribute-specifier-seq_opt
    expression , assignment-expression
```

We can also special-case parenthesized expressions, so that their attribute applies to the inner expression.

With `[[discard]]`, this would allow for instance this:

```
[[discard]] f();  // no warning, attribute on statement
f() [[discard]];  // ditto, equivalent, attribute on the expression

[[discard]] a(), b();  // no warnings, attribute on statement
a(), b() [[discard]];  // no warnings, attribute on the entire expression
a() [[discard]], b();  // no warning for a(), possible warning for b()

int x = (a() [[discard]], b()); // no warning; suppressed for a(), and b()

struct S {
    S(int i)
        : m_i((check(i) [[discard]], i)) {}   // no warning

    int m_i;
};
```

Note that, for the moment being, in this approach we are proposing attributes on the *expression* grammar production, and not attributes on **all** possible kinds of sub-expressions. We believe that complicating the grammar to allow for attributes "everywhere" is not worth the effort, because one can always wrap a subexpression in parenthesis in order to apply an attribute to it. However, we also believe that this approach does not impede such an extension in the future.

Here are some more examples:

```
int a[10];

a[1] = x + y [[attr]];    // attr applies to `a[1] = x + y`
a[2] = x + (y [[attr]]);  // attr applies to `y`
a[3] = ((x+y) [[attr]]);  // attr applies to `x+y`
a[4] = (x+y [[attr]]);    // attr applies to `x+y`

// Attributes can only be applied on expressions, and not (unparenthesized)
//   assignment-expressions, primary-expressions, etc.:
a[5] = x [[attr]] + y;    // ill-formed
a[i [[attr]] ] = 42;      // ill-formed
a[6] [[attr]] = 123;      // ill-formed

x [[attr]] = -1;          // ill-formed

int x = [[attr]] f();     // ill-formed
int y = f() [[attr]];     // ill-formed (the initializer wants an assignmer
int z = (f() [[attr]]);   // OK: attr applies to `f()`


// We can apply attributes to arbitrary sub-expressions by parenthesizing t
// attr1 applies to `x`
// attr2 applies to `y+2`
// attr3 applies to the whole expression
(x [[attr1]]) = (y+2 [[attr2]]) [[attr3]];


// attr1 applies to `c.foo()`
// attr2 applies to `*c`
// attr3 applies to the whole requires-expression
template <typename T>
concept C = (requires (C c) {
    c.foo() [[attr1]];
    { (*c) [[attr2]] } -> convertible_to<bool>;
} [[attr3]]);


// attr1 applies to the statement
// attr2 applies to the closure's function call operator
// attr3 applies to the closure's function call operator's type
// attr4 applies to the overall expression
```

```
[[attr1]] [] [[attr2]] () [[attr3]] {} () [[attr4]];

// attr applies to the closure's function call operator, and not
// to the requires-expression in the requires-clause, as per
// [expr.prim.lambda.general]/3
[]<typename T> requires
    requires (T t) { *t; }
        [[attr]] () {};
```

3.2.1. Problems

The proposed change conflicts with some existing grammar productions. We are aware of at least two.

1. The production(s) for new expressions for arrays, added by [N3033] as resolution of [CWG951]. In [expr.new] there are the following productions:

   ```
   noptr-new-declarator:
       [ expression_opt ] attribute-specifier-seq_opt
       noptr-new-declarator [ constant-expression ] attribute-specifier-se
   ```

   with the attribute appertaining to the associated array type. This means that `auto ptr = (new T[123] [[someattribute]]);` is legitimate code today.

   We are unsure about a use case for allowing attributes specifically on new expressions for arrays. (Rather than applying an attribute on the array type right into the new expression, can't the same intent be better expressed by having an attribute on e.g. a type alias to the array type, while allowing the attribute in `new` to appertain to the expression?)

2. The production(s) for conversion functions in [class.conv.fct], added by [N2761]. A *primary-expression* can contain a *conversion-function-id* as subexpression, and the associated grammar allows attributes at the end:

   ```
   ptr-declarator ( parameter-declaration-clause ) cv-qualifier-seq_opt
     ref-qualifier-seq_opt noexcept-specifier_opt attribute-specifier-seq_opt
   ```

Here the attribute appertains to the function type ([dcl.fct]/1). For instance, this code is legitimate:

```
struct S { operator int() const; };
auto ptr = (&S::operator int [[attribute]]);
```

A similar example is available in [P2173R1].

An implementation-specific attribute can, in principle, be used to select a specific overload (since they apply to the type):

```
// example and explanation courtesy of Richard Smith
struct S {
  operator int() [[vendor::attr1]] const;  // #1
  operator int() [[vendor::attr2]] const;  // #2
};

auto ptr = (&S::operator int [[vendor::attr2]]); // select #2
```

How to solve these cases? **Here we seek EWGI and EWG guidance.**

A possible solution could be to simply enshrine that, in case of an ambiguity, the tie is resolved in favour of the status-quo.

If instead grammar changes for these productions are wanted, unfortunately we are unable to evaluate the real-world breakage that could result.

§ 3.3. Attributes on expressions: approach 2

Instead of allowing attributes on any expression, we may just introduce them on **parenthesized expressions**. In this case we would have room on the **left-hand side**, as there's a token (the open parenthesis) that separates the expression from anything preceding it.

This is the grammar change that is required:

```
primary-expression:
    literal
    this
    ( attribute-specifier-seq_opt expression )
    id-expression
```

```
    lambda-expression
    fold-expression
    requires-expression
```

Here's some examples of attributes on expressions that this approach allows for:

```
int a[10];

[[attr]] a[0] = x + y;    // attr applies to the statement
([[attr]] a[1]) = x + y;  // attr applies to `a[1]`
a[2] = x + y [[attr];     // ill-formed
a[3] = [[attr]] x + y;    // ill-formed
a[4] = ([[attr]] x) + y;  // attr applies to `x`
a[5] = ([[attr]] x + y);  // attr applies to `x + y`
([[attr]] a[6] = x + y);  // attr applies to `a[6] = x + y`


// attr1 applies to the whole requires-expression
// attr2 applies to `c.foo()`
// attr3 applies to `*c`
template <typename T>
concept C = ([[attr1]] requires (C c) {
    ([[attr2]] c.foo());
    { ([[attr3]] *c) } -> convertible_to<bool>;
});


// attr1 applies to the statement
// attr2 applies to the overall expression
// attr3 applies to the closure's function call operator
// attr4 applies to the closure's function call operator's type
[[attr1]] ([[attr2]] [] [[attr3]] () [[attr4]] {} ());
```

Specifically for attribute `[[discard]]`, this approach leads to this syntax/semantics:

```
[[discard]] f();   // no warning, attribute on statement
([[discard]] f()); // no warning, attribute on expression
f() [[discard]];   // ill-formed

[[discard]] a(), b();     // no warnings, attribute on statement
([[discard]] a(), b());   // no warnings, attribute on the entire expressio
```

```
(([[discard]] a()), b();   // no warning for a(), possible warning for b()

int x = ([[discard]] a(), b()); // no warning; suppressed for a(), and b()
int y = ([[discard]] a()), b(); // no warning; ditto

struct S {
    S(int i)
        : m_i(([[discard]] check(i)), i)) {}   // no warning

    int m_i;
};
```

From purely an aesthetic point of view, having the attribute on the left-hand of the expression that it appertains to may feel more more "natural"; expressions are usually read left-to-right, and having the attribute at the very end (like in approach 1) may be surprising.

On the other hand, this approach requires parenthesizing all the expressions that we want to mark with an attribute. Given the relatively rarity of such expressions, the trade-off of the extra syntax can be very acceptable. Again, compared with approach 1, it makes it more clear which sub-expressions are affected by the attribute.

Again, we seek EWGI and EWG guidance here.

We are not aware of any conflicts in the grammar for this approach (if there were, they would already by conflicting with the grammar for statements).

§ 3.4. Can `void` be discarded?

Yes. We believe that such a situation can happen in practice, for instance in generic code, and such a restriction sounds therefore unnecessary and vexing.

§ 3.5. What should happen if `[[discard]]` is applied to an expression which isn't a discarded-value expression?

(Or, similarly, applied to an *expression-statement* whose expression isn't discarded-value.)

For example, using the syntax from approach 1:

```
[[nodiscard]] int f();

// not *actually* discarding:
int a = (f() [[discard]]);
```

Should we accept or forbid these usages, as the attribute is meaningless (at best) or misleading (at worst)? For the moment being, we are proposing to **accept** the code, under the rationale that the attribute serves to suppress a `[[nodiscard]]` warning. Since the warning would not be generated, there is nothing to suppress. Implementations can still diagnose these usages as QoI.

✧ ✧ ✧

There is also the broader issue of expressions marked as `[[discard]]` that contain discarded-value subexpressions:

```
// f()'s return value is discarded, but g()'s is not
int a = ((f(), g()) [[discard]]);
```

In the above example the attribute applies to the entire comma expression. Should the `f()` discarded-value subexpression be diagnosed? We propose that the warning should be suppressed even in this case, as the expression is part of a broader one marked as `[[discard]]` (in other words, that the user was OK with the idea of discarding values somewhere *inside* that expression).

§ 4. Impact on the Standard

This proposal is a core language extension. It proposes:

- changes to the C++ grammar to allow attributes on expressions;
- a new standard attribute, spelled `[[discard]]` or `[[discard("with reason")]]`, to mark expressions or statements whose result we want to expressly discard.

No changes are required in the Standard Library.

§ 5. Technical Specifications

All the proposed changes are relative to [N4958].

§ 5.1. Proposed wording

There are some slight changes in wording depending on the approach chosen.

✧ ✧ ✧

Approach 1 only: modify the grammar productions for *expression* in [expr.comma] and in [gram.expr] as shown:

> *expression:*
>
> *assignment-expression* *attribute-specifier-seq*$_{opt}$
>
> *expression* , *assignment-expression*

In [expr.comma], add a new paragraph after paragraph 1:

> 2. The optional *attribute-specifier-seq* appertains to the expression, unless the expression is a parenthesized expression ([expr.prim.paren]), in which case it appertains to the expression between the parentheses.

✧ ✧ ✧

Approach 2 only: modify the grammar productions for *primary-expression* in [expr.prim] and in [gram.expr] as shown:

> *primary-expression*:
>
> *literal*
>
> this
>
> (*attribute-specifier-seq*$_{opt}$ *expression*)
>
> *id-expression*
>
> *lambda-expression*
>
> *fold-expression*
>
> *requires-expression*

In [expr.prim.paren], append a new paragraph:

> 2. The optional *attribute-specifier-seq* appertains to the expression, unless the expression is itself a parenthesized expression, in which case it appertains to the expression between the parentheses.

✧ ✧ ✧

Modify [dcl.attr.grammar]/5 as shown:

> 5. Each *attribute-specifier-seq* is said to *appertain* to some entity , expression or statement, identified by the syntactic context where it appears ([stmt.stmt], [dcl.dcl], [dcl.decl] , [**LINK**]).

With *LINK* being "expr.comma" for approach 1 and "expr.prim" for approach 2.

✧ ✧ ✧

In [cpp.cond] append a new row to Table 21 ([tab:cpp.cond.ha]):

Attribute	Value
`discard`	`YYYYMML`

with `YYYYMML` determined as usual.

✧ ✧ ✧

In [dcl.attr.nodiscard] insert a new paragraph after 3:

> 4. A potentially-evaluated discarded-value expression ([expr.prop]) which is a nodiscard call and which is neither
>
> - explicitly cast to `void` ([expr.static.cast]), or
> - an *expression* (or subexpression thereof) marked with the `discard` attribute ([dcl.attr.discard]), or
> - an expression (or subexpression thereof) of an *expression-statement* marked with the `discard` attribute ([dcl.attr.discard])
>
> is a *discouraged nodiscard call*.

Renumber and modify the existing paragraph 4 as shown:

> ~~4.~~ 5. *Recommended practice*: Appearance of a ~~nodiscard call as a potentially-evaluated discarded-value expression ([expr.prop])~~ discouraged nodiscard call is discouraged ~~unless explicitly cast to `void`~~ . Implementations should issue a warning in such cases. [...]

And renumber the rest of the paragraphs in [dcl.attr.nodiscard].

Modify the Example 1 as shown (including only the lines for the chosen approach):

```
struct [[nodiscard]] my_scopeguard { /* ... */ };
struct my_unique {
  my_unique() = default;                                  // does not acqui
  [[nodiscard]] my_unique(int fd) { /* ... */ }           // acquires resou
  ~my_unique() noexcept { /* ... */ }                     // releases resou
  /* ... */
};
struct [[nodiscard]] error_info { /* ... */ };
error_info enable_missile_safety_mode();
void launch_missiles();
void test_missiles() {
  my_scopeguard();              // warning encouraged
  (void)my_scopeguard(),        // warning not encouraged, cast to void
    launch_missiles();          // comma operator, statement continues
  my_unique(42);                // warning encouraged
  my_unique();                  // warning not encouraged

  enable_missile_safety_mode(); // warning encouraged
  launch_missiles();


  [[discard]] my_unique(123);               // warning not encouraged
  [[discard("testing")]] my_unique(-1);     // warning not encouraged
  [[discard]] my_unique();                  // warning not encouraged

  [[discard]] my_unique(),
    enable_missile_safety_mode();           // warning not encouraged

  // approach 1:
  my_scopeguard() [[discard]],
    launch_missiles();                      // warning not encouraged

  my_scopeguard() [[discard]],              // warning not encouraged
    my_unique(42);                          // warning encouraged

  // approach 2:
  ([[discard]] my_scopeguard()),
    launch_missiles();                      // warning not encouraged

  ([[discard]] my_scopeguard()),            // warning not encouraged
    my_unique(42);                          // warning encouraged
```

```
}
error_info &foo();
void f() { foo(); }            // warning not encouraged: not a nodiscar
                               // the (reference) return type nor the fu
```

✧ ✧ ✧

Add a new subclause at the end of [dcl.attr], with the following content:

??? Discard attribute [dcl.attr.discard]

1. The *attribute-token* `discard` may be applied to an *expression* or to an *expression-statement*. An *attribute-argument-clause* may be present and, if present, shall have the form:

(*unevaluated-string*)

2. *Recommended practice*: Implementations should suppress the warning associated with a nodiscard call ([dcl.attr.nodiscard]) if such a call is an *expression* (or subexpression thereof) marked as `discard`, or an expression (or subexpression thereof) of an *expression-statement* marked as `discard`. The value of a *has-attribute-expression* for the discard attribute should be `0` unless the implementation can suppress such warnings.

   The *unevaluated-string* in a discard *attribute-argument-clause* is ignored.

   [*Note* 1: the string is meant to be used in code reviews, by static analyzers and in similar scenarios. — *end note*]

§ 6. Acknowledgements

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[CWG951]

Sean Hunt. *Problems with attribute-specifiers*. 5 August 2009. CD2. URL: https://wg21.link/cwg951

[ES.48]

Bjarne Stroustrup; Herb Sutter. *C++ Core Guidelines, ES.48: Avoid casts*. URL: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Res-casts

[N2761]

J. Maurer, M. Wong. *Towards support for attributes in C++ (Revision 6)*. 18 September 2008. URL: https://wg21.link/n2761

[N3033]

Daveed Vandevoorde. *Core issue 951: Various Attribute Issues*. 5 February 2010. URL: https://wg21.link/n3033

[N4958]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 14 August 2023. URL: https://wg21.link/n4958

[P0068R0]

Andrew Tomazos. *Proposal of [[unused]], [[nodiscard]] and [[fallthrough]] attributes*. 3 September 2015. URL: https://wg21.link/p0068r0

[P0189R1]

Andrew Tomazos. *Wording for [[nodiscard]] attribute*. 29 February 2016. URL: https://wg21.link/p0189r1

[P1301R4]

JeanHeyd Meneide, Isabella Muerte. *[[nodiscard("should have a reason")]]*. 5 August 2019. URL: https://wg21.link/p1301r4

[P1771R1]

Peter Sommerlad. *[[nodiscard]] for constructors*. 19 July 2019. URL: https://wg21.link/p1771r1

[P2173R1]

Daveed Vandevoorde, Inbal Levi, Ville Voutilainen. *Attributes on Lambda-Expressions*. 9 December 2021. URL: https://wg21.link/p2173r1

[P2968R0]

Peter Sommerlad. *Make std::ignore a first-class object*. 7 September 2023. URL: https://wg21.link/p2968r0

[WG14-N2267]

Aaron Ballman. *The nodiscard attribute*. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2267.pdf